

Cofferdam

Code management for the AI age

Standardise how AI writes code — with the Credo-style linter rolled up beneath it.

Code is a commodity that agents now generate fast.

Cofferdam is how a team pins down what “**good**” means *once* — and makes every author, human or AI, comply with it *without reading the whole codebase*.

Declare once

The architecture lives in one file:
`cofferdam.invariants.toml`.

Comply before the edit

`advise` pushes the rules to the agent
ahead of the change.

Roll up the linter

The Credo five-category checks are the
floor, not the product.

Positioning: from linter to code-management

Old framing — “Credo for TypeScript”

A TS static analyzer with five prioritized categories.
Competes with ESLint on rule count.
Reacts *after* code is written.

Comparison set: ESLint, tslint — a crowded race to the bottom.

New framing — “code management for the AI age”

The substrate for an organisation’s code standards.
Agents comply against the spec *before* they write.
Linting is one rolled-up layer.

Comparison set: Credo, Semgrep, fitness functions — and the AI toolchain.

“Cofferdam isn’t a linter, it’s a code management tool.” — design-principles.md

The IP — four defensible claims

1 • Spec as shared truth

Architecture declared once, read by many checks *and* by agents. One contract, N enforcement points — vs a linter's per-rule config that drifts.

2 • Agents comply before they write

`advise <file>` pushes applicable rules ahead of the edit; `advise --diff` pre-flights a change (`would_fire` / `would_clear`). Every other linter only reacts *after*.

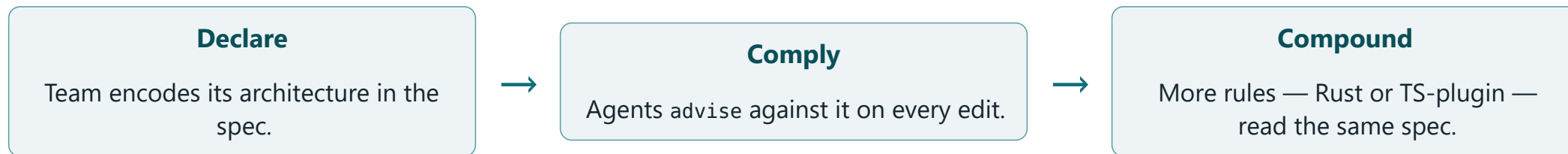
3 • Cross-file architectural reasoning

Canonical graph + typed corpus → layer violations, import cycles, orphan / dead exports, frozen boundaries, scripted invariants. The leapfrog over Credo / ESLint.

4 • Polylingual substrate

Rules run on a language-agnostic graph; languages plug in via adapters (Rust adapter is the live proof). Opinions outlive the language.

The moat: the standardisation flywheel



Once a team's architecture *and* its agents both depend on the spec, it becomes the canonical contract for all code production. Switching cost climbs; value compounds with every rule added.

The operational discipline — stable IDs, baseline adoption, severity/priority split, suppression-as-metadata, a single clean -- robot channel — is what makes the opinions *trustworthy* for CI and agents alike.

Marketecture I — the loop

● shipped ● in flight ○ not started

THE SPEC · `cofferdam.invariants.toml` — declared once, read by many

- layers + allow
- public_api
- boundaries (frozen)
- invariants (forbid / require)
- scripted predicate DSL
- schema versioning

↓ read by three audiences ↓

Humans

One file *is* the architecture — not folklore. An architect reads it on day one.

Agents · the AI-age surface

- `advise <file>` — rules before the edit
- `advise --diff` — pre-flight a change
- `--robot JSON` — clean data channel
- type-aware checks (ts-morph)
- `cofferdam-mcp` — native MCP tool (cd-9r3)

The engine

- discovery + parse
- 3-phase Check contract
- corpus + finalize
- incremental cache

Marketecture II — the stack

● shipped ● in flight ○ not started

FOUR LEVERAGE LAYERS · narrow → wide

● built-in checks → ● per-check options + per-glob [[overrides]] (cd-m5tu, 0.3.5) → ● invariants spec (*the centerpiece*) → ● plugin
SDK @cofferdam/check-sdk (cross-file corpus)

Polylingual substrate · the frontier

- canonical graph schema
- Rust adapter proof
- Span→Location refactor
- LSP server over stdio
- adapter contract: any artifact → graph (cd-9hp.10)
- Location variant rendering + TS SDK (cd-0gwd)

Distribution

- GH binary releases + npm wrapper
- cofferdam-check GitHub Action
- SARIF → GitHub Code Scanning
- baseline · --since · severity gate
- VitePress docs + generated catalog

The Credo floor — 27 checks, rolled up as one layer

Warning · 6

- TripleEquals
- NoConsoleLog
- NoDebugger
- NoEval
- UnusedImport
- UnusedNullCheck

Refactor · 8

- CyclomaticComplexity
- CognitiveComplexity
- DuplicateBlock
- UnusedVariable
- PreferOptionalChain
- PreferNullishCoalescing
- DeadExport
- LongAndComplex

Design · 8

- LayerViolation
- OrphanExport
- BoundaryFrozen
- InvariantViolation
- ImportCycle
- DuplicateExportName
- MaxParameters
- ScriptedInvariant

Readability · 2

- MaxLineLength
- MaxFunctionLength

Consistency · 3

- QuoteStyle
- UnusedSuppression
- BroadSuppression

Five categories, priority-sorted within each — the Credo discipline. Configurable by option, by glob, or disabled entirely. This is the *floor*, not the product.

Status — the spine is done; the frontier remains

Done — 193 / 200 beads

- AI surface: `advise`, `advise --diff`, `--robot`, type-aware
- Spec: invariants + predicate DSL + schema versioning + e2e fixtures
- Architecture epic `cd-9hp`: 11 / 12
- Plugin SDK + cross-file corpus
- Canonical graph · Rust adapter · LSP
- Per-glob `[[overrides]]` (shipped 0.3.5)
- 27 checks · SARIF + 3 formatters · full distribution

Left to do — 5 beads, none blocking

- **cd-9hp.10** (P3) adapter contract → graph
the last unredeemed architecture pledge
- **cd-0gwd** (P3) Location rendering + TS SDK exposure
- **cd-9r3** (P4) `cofferdam-mcp` native MCP server
finishes the AI-context-substrate story
- **cd-i255** (P4) lint Typst packages (dogfood polylingual)
- **cd-mhks** (P3) flaky timing bench tests

Where it's heading

The linter is shipped. The spec + agent-advisory loop is shipped.
What remains turns cofferdam from "Credo-for-TS with an AI surface" into a *polylingual standardisation substrate*: the adapter contract makes the rules language-agnostic; the MCP server makes the agent surface a first-class tool.